MiniShop - soutenance SAÉ 302

Application de vente en ligne en PHP - 12 diapositives - 20 min + questions

- Une application **100 % PHP / MySQL**, sans framework ni dépendance runtime
- Un **cahier des charges** de 16 sections, des règles de gestion numérotées RB-01 -> RB-20
- **Une base de démonstration** livrée : 36 produits (3 pages de 12), 3 clients, un back-office

> Tout ce qui est annoncé dans ce diaporama est exécutable depuis le dépôt.

Le besoin

Un mini-shop crédible, pas une maquette

- Vendre du matériel informatique : catalogue, panier, commande suivie, back-office
- **14 cas d'utilisation** (UC-01 -> UC-14), 3 acteurs : visiteur/client, gestionnaire, super-administrateur
- Contraintes imposées : PHP procédural objet sans framework, MySQL/PDO, Git, pas de CMS

> Objectif : un logiciel défendable en soutenance, pas une démonstration jetable.

Périmètre livré

Ce que fait réellement l'application

- **Vitrine** : accueil, catalogue filtrable, fiche produit, recherche, pages légales
- **Client** : compte, panier (sans rechargement), commande, suivi, annulation, changement de mot de passe
- **Back-office** : produits, stocks, commandes, clients, équipe, paramètres (frais de port, TVA)
- **Transverse** : journal d'audit, erreurs 404/403/500 habillées, accessibilité et responsive

> 32 écrans capturables, 1 seul point d'entrée : `public/index.php`.

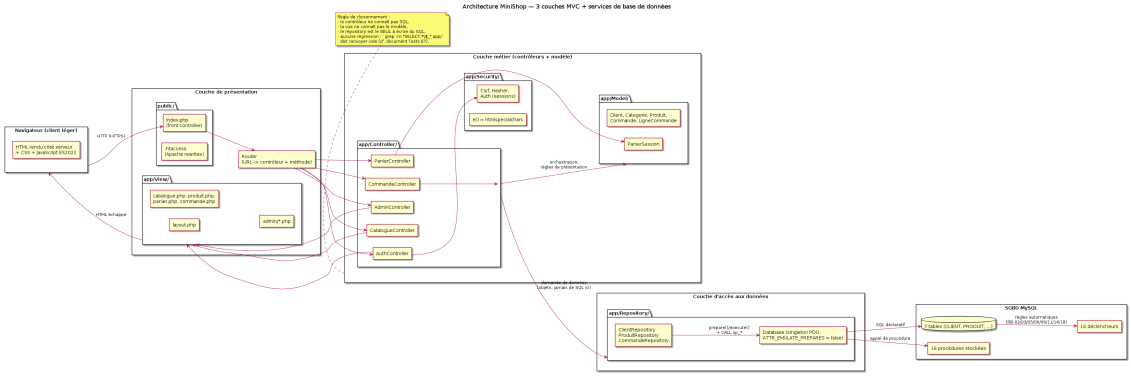
Architecture

MVC strict, front controller, zéro dépendance

- `public/` = seul dossier exposé (front controller, CSS, JS, images)
- `app/Controller` -> `app/Repository` -> magasin de données ; `app/View` sans logique

• Routeur à **liste blanche** : aucune route administrative par convention implicite

• Interface `StoreInterface` : deux implémentations - JSON (démo) et **PDO/MySQL** (production)



Architecture MVC

> Le choix du magasin se fait par configuration, sans toucher aux contrôleurs.

Modèle de données

8 entités, 3 vues, 17 procédures, 16 déclencheurs

• Entités : client, administrateur, catégorie, produit, commande, ligne_commande, historique_statut, paramètre

• Contraintes portées par la base : `CHECK`, clés étrangères, unicité, prix >= 0, stock >= 0

• Déclencheurs : décrémentation de stock, journal de statut, interdiction de suppression d'un produit commandé

• Procédures : création de commande transactionnelle, annulation, alertes de stock, purge

> La cohérence ne repose pas sur l'interface : elle est garantie par le SGBD.

Règles de gestion

20 règles, testées une par une

• RB-01 à RB-20 : prix TTC calculé côté serveur, plafonnement au stock, frais de port offerts, transitions d'état, suppression logique

• **29 tests SQL** : contraintes, déclencheurs, procédures, vues, port, rollback

• Matrice de transitions : `BROUILLON -> EN_PREPARATION -> PAYEE -> EXPEDIEE -> LIVREE`, `ANNULEE` en sortie

> Une règle non testée est une règle qui n'existe pas.

Sécurité

Dix contrôles statiques, exécutés à chaque commit

• Mots de passe **bcrypt**, sessions `HttpOnly` + `SameSite=Lax`, identifiant régénéré à la connexion

• **CSRF** sur tous les formulaires et sur l'API (comparaison `hash_equals`, jeton CSPRNG)

• SQL **préparé** partout (aucune concaténation), sorties échappées par `e()`, CSP stricte

• Contrôle d'accès : route `admin`/`super`, vérification d'appartenance des commandes (IDOR)

> `bash tests/security/controles.sh` -> 10/10 conformes.

Tests

Une pyramide, pas un slogan

• **PHPUnit** : 6 classes, 34 tests (texte, filtres, prix, transitions, magasin JSON, contrats de sécurité)

• **Intégration PHP** : moteur JSON, règles de gestion, unitaires, fonctionnalités UC-01 -> UC-14

• **Contrats** : design system (28 vérifications) et responsive (48 vérifications)

• **E2E HTTP** : 185 vérifications sur l'application complète (cookies, CSRF, refus)

• **CI** : GitHub Actions (`.github/workflows/ci.yml`) ; aucun pipeline GitLab équivalent n'est revendiqué

> `npm run test:all` ▶ tout doit être vert avant chaque commit.

Design

Un système, pas des feuilles de style empilées

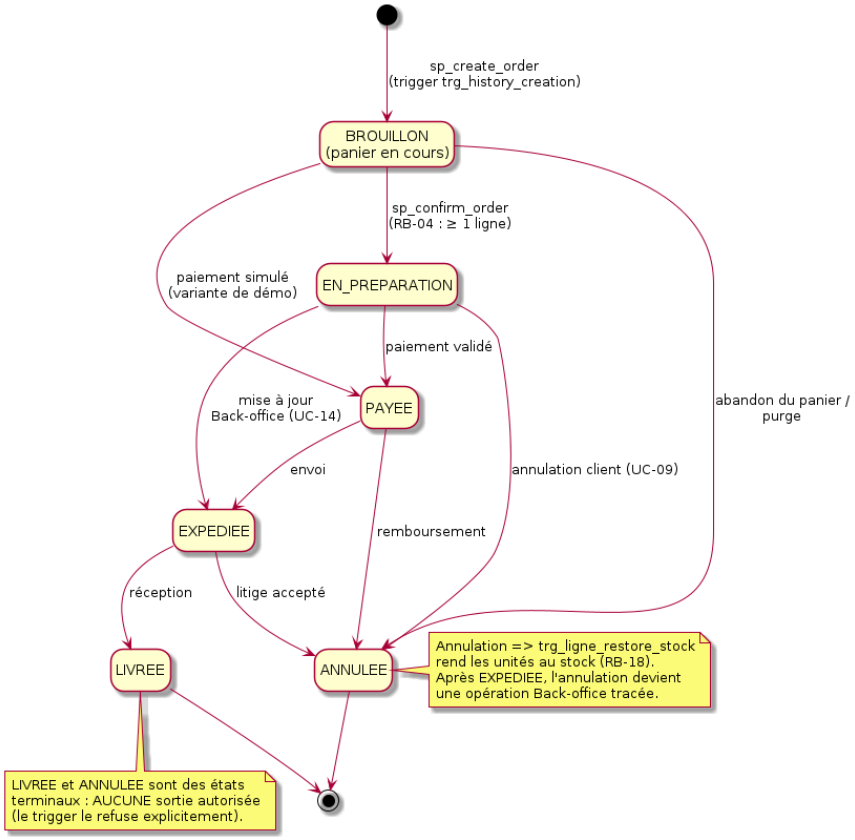
• Étude de 5 concurrents (Amazon, Cdiscount, Fnac, Boulanger, Back Market) -> `docs/DA.md`

• Jetons : palette contrastée AA/AAA, échelle typographique en `rem`, grille 8 px

• Responsive vérifié de **320 px à 1536 px**, paliers en `em`, cibles tactiles >= 44 px

• Accessibilité : lien d'évitement, focus visible, `aria-live`, contrastes mesurés

Diagramme des états d'une COMMANDE (RB-11) — transitions interdites = refusées par trg_commande_transition_statut



États de commande

> Aucun débordement horizontal : l'audit WCAG 1.4.10 est rejouable par script.

Démonstration

Trois minutes, trois parcours

• **Client** : recherche -> fiche produit -> ajout au panier (sans rechargement) -> commande -> suivi

• **Gestionnaire** : expédition d'une commande -> journal de statut mis à jour automatiquement

• **Super-admin** : création d'un produit -> contrôle du stock et de l'alerte de rupture

> Comptes livrés : `alice@example.com` / `Demo2026!` et `admin@minishop.fr` / `Admin2026!`.

Difficultés et arbitrages

Ce que nous avons appris

• Deux magasins (JSON et PDO) derrière une même interface : la démo reste possible sans

serveur MySQL

• Le bac à sable sans PHP natif a imposé un runtime WebAssembly : les tests sont identiques sur machine réelle

• Accessibilité et tableaux de données : le défilement d'un tableau est admis par WCAG 1.4.10, à condition de le documenter

• Arbitrages assumés : pas de paiement réel, images produits générées, volumétrie limitée

> Un arbitrage écrit et justifié vaut mieux qu'une fonctionnalité à moitié faite.

Bilan et suites

Où en est le projet

• Application **livrée et testée** : 6 suites + E2E, contrôles de sécurité, design system

• Documentation complète : cahier des charges, tests, conception, support de soutenance, dossier d'architecture

• Écarts restants suivis publiquement dans `docs/10-todo-corrections.md`

• Suites : rendu documentaire automatisé en PDF/DOCX, bascule PDO par défaut, couverture de code

> Questions ?